

CodroidCPP SDK 手册

版本: 2.1.2 | 命名空间: `Codroid`

目录

#	章节	说明
1	快速上手	构建、连接并运行第一个程序
2	核心概念	生命周期、TCP 模型、单位约定、异常处理
3	CodroidClient API	CodroidClient 完整 API 参考
4	运动 API	JointPoint、CartesianPoint、MoveInstruction、MotionWaitOptions、枚举
5	数据类型与枚举	CommandResult、ClientRealtimeState、RobotFrame、异常类
6	CRI 实时数据与控制	CriRealtimeDispatcher、TrajectoryGenerator、TrajectoryRequest
7	IO 与寄存器	DI/DO/AI/AO 操作、寄存器读写
8	辅助工具	主题订阅、全局变量、运动学、ConsoleUtf8

环境要求

平台	编译器	构建工具
Linux	GCC 9+ / Clang 10+	CMake 3.14+
Windows	MSVC 2019/2022/2026	CMake 3.14+
Windows	MinGW-w64 (MSYS2)	CMake 3.14+

依赖项

- **Asio** (独立版, 非 Boost) — 网络通信
- **nlohmann/json** — JSON 序列化
- **GoogleTest** (可选) — 单元测试

构建

Linux

```
chmod +x build_linux.sh
./build_linux.sh
```

产物在 `build_linux/` (包含 `libCodroid.so` 与示例程序)。

Windows (MSVC)

```
build_msvc.bat
```

脚本支持：

- 1：Visual Studio 2019
- 2：Visual Studio 2022
- 3：Visual Studio 2026

产物在 build_msvc/Debug 与 build_msvc/Release。

Windows (MinGW)

```
build_mingw.bat
```

产物在 build_mingw/。

API 命名约定

所有公共方法使用 **PascalCase** 命名，与 C# 和 Python SDK 保持一致。

```
// 正确
robot.ConnectRemoteAndSwitchOn("192.168.8.136");
int di = robot.GetDi(0);
robot.MovJ(joints, 40, 100);
```

单位约定

层级	线性	角度
SDK 公共 API	mm	deg（度）
TCP JSON 协议	mm	deg
CRI UDP 二进制（线路层）	m	rad（弧度）
ClientRealtimeState（已解析）	mm	deg

CriRealtimeDispatcher 在 convert_to_si=true（默认）时会自动将 mm/deg 转换为 m/rad。

固件要求

本 SDK 所有对外接口均要求控制器固件 $\geq 2.3.3.43$ 。

代码常量见 Codroid::MinControllerFirmware（CodroidDefine.h）。

快速上手

构建 SDK

Linux

```
# 克隆仓库
git clone <repository-url>
cd CodroidCPP

# 构建
chmod +x build_linux.sh
./build_linux.sh
```

产物在 `build_linux/`：

- `libCodroid.so` — 动态库
- `examples/` — 示例程序

Windows (MSVC)

```
REM 克隆仓库后
cd CodroidCPP

REM 构建
build_msvc.bat
```

选择 Visual Studio 版本：

- `1`：Visual Studio 2019
- `2`：Visual Studio 2022
- `3`：Visual Studio 2026

产物在 `build_msvc/Debug` 和 `build_msvc/Release`。

Windows (MinGW)

```
cd CodroidCPP
build_mingw.bat
```

产物在 `build_mingw/`。

最小示例

连接控制器，读取数字输入，写入数字输出，然后断开。

```
#include "Codroid/client.hpp"
```

```
#include <iostream>

int main() {
    Codroid::CodroidClient robot;

    // 连接、切换远程、上电
    if (!robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150")) {
        std::cerr << "连接失败" << std::endl;
        return 1;
    }

    // 读取 DI 端口 0
    int di0 = robot.GetDi(0);
    std::cout << "DI 0 = " << di0 << std::endl;

    // 将 DI 值写入 DO 端口 10
    robot.SetDo(10, di0);

    // 断开连接
    robot.Disconnect();
    return 0;
}
```

完整工作流示例

```
#include "Codroid/client.hpp"
#include "Codroid/cri_realtime_dispatcher.hpp"
#include "Codroid/trajectory_generator.hpp"
#include <iostream>

int main() {
    // Windows 控制台 UTF-8 支持
#ifdef _WIN32
    Codroid::ConsoleUtf8::InitConsoleUtf8();
#endif

    Codroid::CodroidClient robot;

    // 1. 连接
    if (!robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150")) {
        std::cerr << "连接失败" << std::endl;
        return 1;
    }

    // 2. IO 操作
    int di0 = robot.GetDi(0);
    robot.SetDo(10, di0);

    // 3. 寄存器
    double regValue = robot.GetRegisterValue(49100);
    robot.SetRegisterValue(49100, regValue + 1);
}
```

```

// 4. 运动
auto joints = Codroid::JointPoint::Degrees({0, 0, 90, 0, 90, 0});
robot.MovJ(joints, 40, 100);

// 5. 阻塞运动
auto state = robot.GetRobotRealtimeState();
auto target = Codroid::CartesianPoint::MmDegWithRef(
    {400, 0, 300, 180, 0, 0},
    state.joint_position
);
robot.MovLSync(target, 150, 500);

// 6. 断开
robot.Disconnect();
return 0;
}

```

运行示例程序

```

# Linux
cd build_linux
./examples/01_basic_usage 192.168.8.136

# Windows (MSVC)
cd build_msvc\Release
01_basic_usage.exe 192.168.8.136

```

错误处理

TCP 指令失败时的行为取决于 `SetThrowOnCommandError` 设置：

默认模式（不抛异常）

```

Codroid::CodroidClient robot;
robot.Connect("192.168.8.136");

auto result = robot.SetDo(999, 1); // 无效端口
if (!result.Ok()) {
    std::cerr << "控制器错误: " << result.error_msg << std::endl;
}

```

抛异常模式

```
robot.SetThrowOnCommandError(true);

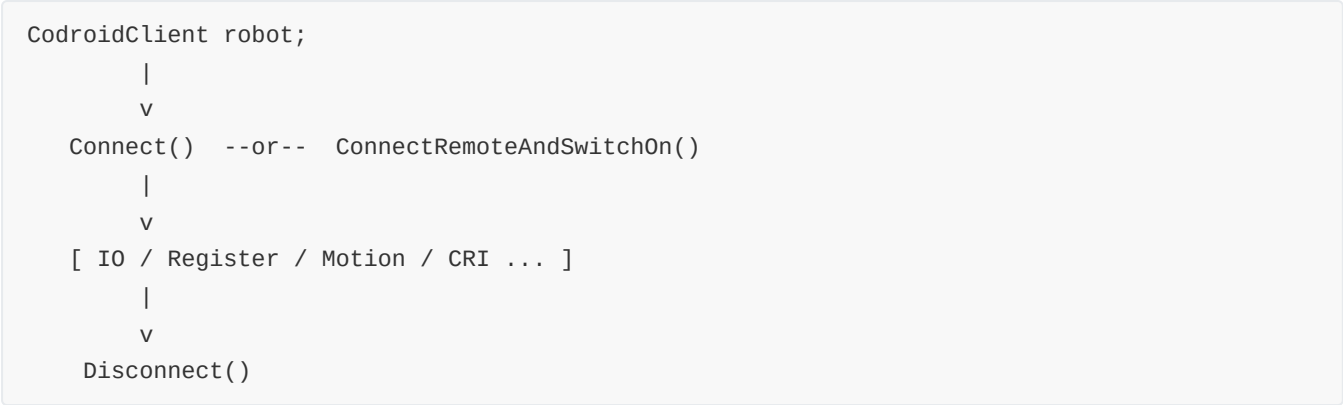
try {
    robot.SetDo(999, 1); // 无效端口
} catch (const Codroid::CodroidCommandException& ex) {
    std::cerr << "控制器错误: " << ex.controller_error() << std::endl;
}
```

异常类型

异常	条件
CodroidCommandException	控制器返回 err 字段
CodroidException	通用运行时错误
std::runtime_error	标准库异常

核心概念

CodroidClient 生命周期



```
Codroid::CodroidClient robot;

try {
    robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150");
    // ... 使用 robot ...
} catch (...) {
    robot.Disconnect(); // 始终调用
    throw;
}
robot.Disconnect();
```

构造函数

```
Codroid::CodroidClient robot;
```

- 默认构造，不连接
- 调用 `Connect` 或 `ConnectRemoteAndSwitchOn` 建立连接
- TCP 端口默认 **9001**

属性

方法	返回类型	说明
<code>GetRobotRealtimeState()</code>	<code>ClientRealtimeState</code>	CRI 数据快照的线程安全副本
<code>GetCriUdpListenPort()</code>	<code>int</code>	CRI UDP 监听端口

回调

```
robot.SetCriDataReceived([](const Codroid::ClientRealtimeState& data) {
    std::cout << "关节: ";
    for (auto j : data.joint_position) std::cout << j << " ";
    std::cout << std::endl;
});
```

每当解析完一个有效的 CRI UDP 帧后触发。回调在内部接收线程上执行，避免长时间阻塞。

TCP 指令模型

每个与控制器通信的 SDK 方法都遵循以下模式：

- 1. SDK 分配唯一的 `id`
- 2. SDK 将 `{ id, ty, db }` 序列化为 JSON 并通过 TCP 发送
- 3. 控制器响应 `{ id, ty, db, err }`
- 4. SDK 通过 `id` 匹配响应
- 5. 若 `err` 非空则 `CommandResult::error_msg` 非空（或抛异常）
- 6. 若 10 秒内未收到响应则超时

CommandResult

```
struct CommandResult {
    int id = 0;                // 请求 id
    std::string ty;            // 响应类型
    std::string error_msg;     // 错误信息（空表示成功）
    std::string raw_json;      // 完整响应 JSON

    bool ok() const noexcept;  // error_msg 为空返回 true
};
```

大多数方法返回 `CommandResult`。检查 `ok()` 判断是否成功。

单位约定

SDK 公共 API 使用毫米和度。这与 TCP JSON 协议一致。

上下文	线性	角度
SDK API、TCP JSON	mm	deg
CRI UDP 线路格式	m	rad
<code>ClientRealtimeState</code> （已解析）	mm	deg

重要: CRI UDP 二进制载荷使用米和弧度。SDK 在内部自动转换为 mm/deg。不要假设原始 UDP 浮点数是 mm/deg。

命名约定

所有公共方法使用 **PascalCase**，与 C# 和 Python SDK 保持一致。

```
robot.ConnectRemoteAndSwitchOn("192.168.8.136");
int di = robot.GetDi(0);
robot.MovJ(joints, 40, 100);
robot.SetDo(10, 1);
```

线程安全

- `GetRobotRealtimeState()` — 线程安全（返回副本）
- `SetCriDataReceived(callback)` — 线程安全
- 所有 TCP 方法 — 可从任意线程调用，但不要在同一 `CodroidClient` 上并发调用
- `CriRealtimeDispatcher::SendCommand` / `SendTrajectory` — 非线程安全

异常类型

异常	触发条件	来源
<code>CodroidCommandException</code>	控制器返回 <code>err</code> 字段	TCP 响应
<code>CodroidException</code>	通用运行时错误	SDK 内部
<code>std::runtime_error</code>	标准库异常	标准库

CodroidCommandException 属性

```
class CodroidCommandException : public CodroidException {
public:
    int request_id() const noexcept;           // 协议请求 ID
    const std::string& command_ty() const noexcept; // 如 "Robot/move"
    const std::string& controller_error() const noexcept; // 控制器的 err 字段
    const std::string& raw_response_json() const noexcept; // 完整响应
};
```

CodroidClient API 参考

类: `CodroidClient`
命名空间: `Codroid`
头文件: `Codroid/client.hpp`

构造与析构

CodroidClient()

默认构造函数，不建立连接。

```
Codroid::CodroidClient robot;
```

~CodroidClient()

析构函数。如果未调用 `Disconnect()`，析构时会自动清理资源。

连接管理

Connect

```
bool Connect(const std::string& ip, int port = 9001);
```

建立 TCP 连接，不切模式、不自动上电。

参数	类型	说明
<code>ip</code>	<code>string</code>	控制器 IP 地址
<code>port</code>	<code>int</code>	TCP 端口，默认 9001

返回: 连接成功返回 `true`

ConnectRemoteAndSwitchOn

```
bool ConnectRemoteAndSwitchOn(const std::string& ip, int port = 9001, std::string local_ip = {});
```

连接并执行远程上电/模式切换。`local_ip` 非空时用于 `StartCriDataPush` 绑定本机 UDP。

参数	类型	说明
<code>ip</code>	<code>string</code>	控制器 IP 地址

参数	类型	说明
<code>port</code>	<code>int</code>	TCP 端口，默认 9001
<code>local_ip</code>	<code>string</code>	本机 IP，用于 CRI UDP 绑定

返回: 成功返回 `true`

推荐: 大多数场景使用此方法。

Disconnect

```
void Disconnect();
```

断开 TCP 连接，停止 CRI 相关线程与缓存。

必须调用: 在程序结束前调用，确保资源释放。

NextRequestId

```
int NextRequestId();
```

生成下一个单调递增的请求 ID。多线程发令时避免重复。

模式控制

SwitchOn / SwitchOff

```
CommandResult SwitchOn(int id = 1);
CommandResult SwitchOff(int id = 1);
```

机器人上电 / 下电。

ToManual / ToAuto / ToRemote

```
CommandResult ToManual(int id = 1);
CommandResult ToAuto(int id = 1);
CommandResult ToRemote(int id = 1);
```

切换到手动/自动/远程模式。

ClearSystemError

```
CommandResult ClearSystemError(int id = 1);
```

清除系统错误。

EnterManualModeViaAuto / EnterRemoteModeViaAuto

```
CommandResult EnterManualModeViaAuto(int id = 1);  
CommandResult EnterRemoteModeViaAuto(int id = 1);
```

先切自动再切手动/远程。

ToSimulation / ToActual

```
CommandResult ToSimulation(int id = 1);  
CommandResult ToActual(int id = 1);
```

进入仿真/实机模式。

StartDrag / StopDrag

```
CommandResult StartDrag(int id = 1);  
CommandResult StopDrag(int id = 1);
```

进入/退出拖拽示教模式。

IO 操作

GetDi / GetDo / GetAi / GetAo

```
int GetDi(int port, int id = 1);  
int GetDo(int port, int id = 1);  
double GetAi(int port, int id = 1);  
double GetAo(int port, int id = 1);
```

读取数字量/模拟量输入/输出。

SetDo / SetAo

```
CommandResult SetDo(int port, int value, int id = 1);  
CommandResult SetAo(int port, double value, int id = 1);
```

设置数字量/模拟量输出。

寄存器操作

GetRegisterValue / GetRegisterValues

```
double GetRegisterValue(int address, int id = 1);  
std::vector<ClientRegisterInfo> GetRegisterValues(const std::vector<int>& addresses, int id  
= 1);
```

读取寄存器值。

SetRegisterValue

```
CommandResult SetRegisterValue(int address, double value, int id = 1);
```

设置寄存器值。

运动控制

MovJ / MovL / MovC / MovCircle

```
CommandResult MovJ(const ClientJointPoint& target, double speed, double acceleration, int id  
= 1);  
CommandResult MovJ(const ClientCartesianPoint& target, double speed, double acceleration,  
int id = 1);  
CommandResult MovL(const ClientCartesianPoint& target, double speed, double acceleration,  
...);  
CommandResult MovC(const ClientCartesianPoint& middle, const ClientCartesianPoint& target,  
...);  
CommandResult MovCircle(const ClientCartesianPoint& middle, const ClientCartesianPoint&  
target, int circle_num, ...);
```

运动指令。

Move / MovePath

```
CommandResult Move(const std::vector<ClientMoveInstruction>& path, int id = 1);
CommandResult MovePath(const std::vector<ClientMoveInstruction>& path, int id = 1);
```

多段路径执行。

阻塞运动（Sync）

```
bool MoveSync(const std::vector<ClientMoveInstruction>& path, const MotionWaitOptions& wait = {});
bool MovJSync(const ClientJointPoint& target, double speed, double acc, const MotionWaitOptions& wait = {});
bool MovLSync(const ClientCartesianPoint& target, double speed, double acc, const MotionWaitOptions& wait = {});
bool MovCSync(const ClientCartesianPoint& middle, const ClientCartesianPoint& target, ...);
bool MovCircleSync(const ClientCartesianPoint& middle, const ClientCartesianPoint& target, int circle_num, ...);
```

阻塞式运动，等待 CRI 确认到达目标。

使用前须知: 调用 `*Sync` 方法前，必须确保 CRI 数据已开始推送（调用 `StartCriDataPush` 并等待首帧）。

PauseRobotMotion / ResumeRobotMotion / StopRobotMove

```
CommandResult PauseRobotMotion(int id = 1);
CommandResult ResumeRobotMotion(int id = 1);
CommandResult StopRobotMove(int id = 1);
```

暂停、恢复、停止运动。

MoveTo（规划运动）

MoveTo

```
CommandResult MoveTo(const MoveToParams& params, int id = 1);
```

MoveTo 预设/规划运动。

MoveToHeartbeat

```
CommandResult MoveToHeartbeat(int id = 1);
```

MoveTo 心跳。使用 `MoveToType::Joint` 或 `Line` 时，每 $\geq 500\text{ms}$ 调用一次。

StopMoveTo

```
CommandResult StopMoveTo(int id = 1);
```

停止 MoveTo 运动。

Jog（点动）

Jog

```
CommandResult Jog(const JogParams& params, int id = 1);
```

点动。

StopJog / JogHeartbeat

```
CommandResult StopJog(int id = 1);  
CommandResult JogHeartbeat(int id = 1);
```

停止点动 / 点动心跳。

CRI 实时控制

StartCriDataPush / StopCriDataPush

```
CommandResult StartCriDataPush(const std::string& udpIp, int udpPort, int id = 1);  
CommandResult StopCriDataPush(int id = 1);
```

启动/停止 CRI 数据推送。

StartCriControl / StopCriControl

```
CommandResult StartCriControl(int filterType, int durationMs, int startBuffer, int id = 1);  
CommandResult StopCriControl(int id = 1);
```

启动/停止实时控制会话。

GetRobotRealtimeState

```
ClientRealtimeState GetRobotRealtimeState() const;
```

获取线程安全的 CRI 数据快照。

SetCriDataReceived

```
void SetCriDataReceived(std::function<void(const ClientRealtimeState&)> cb);
```

设置 CRI 数据回调。

WaitForCriData

```
void WaitForCriData(double timeout_s = 5.0);
```

阻塞等待第一个 CRI 数据帧到达。

运动学

ForwardKinematics / InverseKinematics

```
std::vector<double> ForwardKinematics(const FKParams& params, int id = 1);  
std::vector<double> InverseKinematics(const IKParams& params, int id = 1);
```

正解/逆解。

CalculateRelativePose

```
std::vector<double> CalculateRelativePose(const RelativePoseParams& params, int id = 1);
```

笛卡尔相对位姿计算。

全局变量

GetGlobalVars / SaveGlobalVars / RemoveGlobalVars

```
nlohmann::json GetGlobalVars(int id = 1);  
CommandResult SaveGlobalVars(const std::map<std::string, Variable>& vars, int id = 1);  
CommandResult RemoveGlobalVars(const std::vector<std::string>& names, int id = 1);
```

全局变量操作。

主题订阅

SubscribePublishTopic

```
ClientPublishSubscription SubscribePublishTopic(  
    std::string topicTy,  
    std::function<void(const ClientPublishNotification&)> handler,  
    int tc_milliseconds = 100);
```

订阅推送主题。

工程/脚本

RunScript / EnterRemoteScriptMode / Run / RunByIndex / RunStep

```
CommandResult RunScript(const std::string& mainScript, ...);  
CommandResult EnterRemoteScriptMode(int id = 1);  
CommandResult Run(const std::string& projectId, int id = 1);  
CommandResult RunByIndex(int index, int id = 1);  
CommandResult RunStep(const std::string& projectId, int id = 1);
```

工程/脚本操作。

PauseProject / ResumeProject / StopProject

```
CommandResult PauseProject(int id = 1);  
CommandResult ResumeProject(int id = 1);  
CommandResult StopProject(int id = 1);
```

暂停、恢复、停止工程。

机器人设置参数

GetRobotParameters

```
ClientRobotParameters GetRobotParameters(int id = 1);
```

获取设置界面参数。

SetDefaultPayloadId / SetDefaultToolId / SetDefaultUserCoordinateId

```
CommandResult SetDefaultPayloadId(int payloadId, int id = 1);
CommandResult SetDefaultToolId(int toolId, int id = 1);
CommandResult SetDefaultUserCoordinateId(int coordinateId, int id = 1);
```

设置默认负载、工具、用户坐标系编号（1~15）。

SaveToolFrames / SetToolFrame / SavePayloadFrames / SetPayloadFrame / SetUserCoordinateFrame

```
CommandResult SaveToolFrames(const std::vector<ClientRobotFrame>& frames, int id = 1);
CommandResult SetToolFrame(int frame_id, const ClientRobotFrame& frame, int id = 1);
CommandResult SavePayloadFrames(const std::vector<ClientRobotPayload>& frames, int id = 1);
CommandResult SetPayloadFrame(int frame_id, const ClientRobotPayload& frame, int id = 1);
CommandResult SetUserCoordinateFrame(int frame_id, const ClientRobotFrame& frame, int id = 1);
```

保存/修改坐标系。

SetPayload / SetManualMoveRate / SetAutoMoveRate / SetCollisionSensitivity

```
CommandResult SetPayload(int payloadId, int id = 1);
CommandResult SetManualMoveRate(int pct, int id = 1);
CommandResult SetAutoMoveRate(int pct, int id = 1);
CommandResult SetCollisionSensitivity(int sensitivity, int id = 1);
```

运行时参数设置。

错误处理设置

SetThrowOnCommandError / ThrowOnCommandError

```
void SetThrowOnCommandError(bool enable);
bool ThrowOnCommandError() const noexcept;
```

设置/获取错误处理模式。

运动 API 参考

点位类型

JointPoint

关节空间目标点（六轴角，单位：度）。

```
struct JointPoint {  
    std::vector<double> jp; // 六轴关节角（度）  
  
    static JointPoint Degrees(std::vector<double> joints_deg);  
};
```

使用示例：

```
auto joints = Codroid::JointPoint::Degrees({0, 0, 90, 0, 90, 0});  
robot.MovJ(joints, 40, 100);
```

CartesianPoint

笛卡尔末端目标点（TCP 位姿：mm + 度）。

```
struct CartesianPoint {  
    std::vector<double> cp; // [x, y, z, rx, ry, rz]  
    std::vector<double> rj; // 逆解参考关节角（度）  
  
    static CartesianPoint MmDeg(std::vector<double> pose_mm_deg);  
    static CartesianPoint MmDegWithRef(std::vector<double> pose_mm_deg, std::vector<double>  
ref_joints_deg);  
};
```

使用示例：

```
// 不指定参考关节  
auto pose1 = Codroid::CartesianPoint::MmDeg({400, 0, 300, 180, 0, 0});  
  
// 指定参考关节（推荐）  
auto state = robot.GetRobotRealtimeState();  
auto pose2 = Codroid::CartesianPoint::MmDegWithRef(  
    {400, 0, 300, 180, 0, 0},  
    state.joint_position  
);
```

MovePoint

路径段中的单个目标点（协议层）。

```
struct MovePoint {
    std::vector<double> jp;
    std::vector<double> cp;
    std::vector<double> rj;
    std::vector<double> ep;

    static MovePoint Joint(JointPoint joint);
    static MovePoint Cartesian(CartesianPoint cart);
};
```

路径指令

ClientMoveInstruction / MoveInstruction

路径中的一段运动指令。

```
struct ClientMoveInstruction {
    ClientMoveType type = ClientMoveType::MovJ;
    double speed = 60.0;
    double acceleration = 150.0;
    double blend = -1.0;
    double relative_blend = -1.0;
    int circle_num = 1;
    ClientMovePoint target;
    ClientMovePoint middle;
    std::vector<double> coor;
    std::vector<double> tool;

    static ClientMoveInstruction MovJ(ClientJointPoint target, double speed, double
acceleration, double blend = -1.0);
    static ClientMoveInstruction MovJ(ClientCartesianPoint target, double speed, double
acceleration, double blend = -1.0);
    static ClientMoveInstruction MovL(ClientCartesianPoint target, double speed, double
acceleration, double blend = -1.0);
    static ClientMoveInstruction MovL(ClientJointPoint target, double speed, double
acceleration, double blend = -1.0);
    static ClientMoveInstruction MovC(ClientCartesianPoint middle, ClientCartesianPoint
target, ...);
    static ClientMoveInstruction MovCircle(ClientCartesianPoint middle, ClientCartesianPoint
target, int circle_num, ...);
};
```

使用示例：

```

auto state = robot.GetRobotRealtimeState();

std::vector<Codroid::ClientMoveInstruction> path = {
    Codroid::ClientMoveInstruction::MovJ(
        Codroid::JointPoint::Degrees({0, 0, 90, 0, 90, 0}), 40, 100),
    Codroid::ClientMoveInstruction::MovL(
        Codroid::CartesianPoint::MmDegWithRef({400, 0, 300, 180, 0, 0},
state.joint_position),
        150, 500)
};
robot.Move(path);

```

运动类型枚举

ClientMoveType / MoveType

```

enum class ClientMoveType {
    MovJ,          // 关节运动
    MovL,          // 直线（笛卡尔）
    MovC,          // 圆弧（经由中间点）
    MovCircle      // 整圆/多圈
};

```

阻塞运动参数

MotionWaitOptions

```

struct MotionWaitOptions {
    double timeout_s = 60.0;           // 整体等待超时（秒）
    double poll_interval_s = 0.05;    // CRI 轮询间隔（秒）
    double cri_stale_timeout_s = 0.5;  // CRI 数据过期判定（秒）
    int settled_samples = 3;           // InMotion=false 连续稳定采样数
    double joint_tolerance_deg = 0.2;  // 关节目标容差（度）
    double cartesian_position_tolerance_mm = 1.0; // 笛卡尔位置容差（mm）
    double cartesian_orientation_tolerance_deg = 1.0; // 笛卡尔姿态容差（度）
};

```

MoveTo 参数

MoveToType

```

enum class MoveToType : int {
    Stop = -1,      // 停止 MoveTo
    Home = 0,       // 回 Home
    Safe = 1,       // 回安全位
    Candle = 2,     // 回 Candle 位
};

```

```
Packing = 3,      // 回 Packing 位
Joint = 4,        // 关节规划到目标
Line = 5,         // 直线规划到目标
ResumePoint = 6

};
```

MoveToParams

```
struct MoveToParams {
    MoveToType type = MoveToType::Home;
    MoveToTarget target;
};
```

Jog 参数

JogMode

```
enum class JogMode {
    Joint = 1,    // 关节点动
    Line = 2      // 直线点动
};
```

JogParams

```
struct JogParams {
    JogMode mode = JogMode::Line;
    double speed = 0.0;      // -1~1 的比例
    int index = 1;           // 轴号或方向
    CoordType coordType = CoordType::User;
    int coordId = 1;
};
```

数据类型与枚举

通信相关

CommandResult

```
struct CommandResult {  
    int id = 0;  
    std::string ty;  
    std::string error_msg;  
    std::string raw_json;  
  
    bool Ok() const noexcept;  
};
```

实时数据

ClientRealtimeState

```
struct ClientRealtimeState {  
    int64_t timestamp_ms = 0;  
    bool data_valid = false;  
    uint16_t status1_raw = 0;  
    uint16_t status2_raw = 0;  
  
    std::vector<double> joint_position;           // 关节位置, 度  
    std::vector<double> joint_velocity;          // 关节角速度, 度/s  
    std::vector<double> tcp_pose;                // [x,y,z,rx,ry,rz], mm + 度  
    std::vector<double> tcp_velocity;            // 线 mm/s, 角 °/s  
    double tcp_linear_velocity_mm_s = 0.0;  
    std::vector<double> joint_output_torque;  
    std::vector<double> joint_external_force;  
    std::vector<double> external_axis_position;  
  
    bool project_running = false;  
    bool project_stopped = false;  
    bool project_paused = false;  
    bool enabling = false;  
    bool not_enabled = false;  
    bool manual_mode = false;  
    bool dragging = false;  
    bool in_motion = false;  
    bool collision_stopped = false;  
    bool in_safety_position = false;  
    bool has_alarm = false;  
    bool simulation_mode = false;  
    bool emergency_stop_pressed = false;  
    bool rescue_mode = false;  
    bool auto_mode = false;
```

```
bool remote_mode = false;
bool realtime_control_mode = false;
uint8_t cri_error_code = 0;
};
```

IO 相关

ClientIoInfo / ClientRegisterInfo

```
struct ClientIoInfo {
    std::string type;
    int port = 0;
    double value = 0.0;
};

struct ClientRegisterInfo {
    int address = 0;
    double value = 0.0;
};
```

机器人设置

ClientRobotFrame / ClientRobotPayload

```
struct ClientRobotFrame {
    int id = 0;
    double x = 0.0, y = 0.0, z = 0.0;
    double a = 0.0, b = 0.0, c = 0.0;
};

struct ClientRobotPayload {
    int id = 0;
    double m = 0.0;
    double mx = 0.0, my = 0.0, mz = 0.0;
};
```


ClientRobotParameters

```
struct ClientRobotParameters {  
    bool valid = false;  
    int default_tool_id = 0;  
    int default_payload_id = 0;  
    int default_coordinate_id = 0;  
    double max_payload = 0.0;  
    std::vector<ClientRobotFrame> tool;  
    std::vector<ClientRobotPayload> payload;  
    std::vector<ClientRobotFrame> coordinate;  
};
```

主题推送

ClientPublishNotification

```
struct ClientPublishNotification {  
    std::string ty;  
    std::string db_json;  
    std::string raw_json;  
};
```

ClientPublishSubscription

```
class ClientPublishSubscription {  
public:  
    void Dispose();  
    bool IsValid() const noexcept;  
};
```

全局变量

Variable

```
struct Variable {  
    std::string val;  
    std::string nm;  
  
    template<typename T>  
    Variable(const T& value, const std::string& note = "");  
};
```

运动学

FKParams / IKParams

```
struct FKParams {
    std::vector<double> jp;
    std::vector<double> coor, tool, ep;
};

struct IKParams {
    std::vector<double> cp;
    std::vector<double> rj;
    std::vector<double> ep;
};
```

RelativePoseParams

```
struct RelativePoseParams {
    std::vector<double> pos;
    std::vector<double> offset;
    CoordType coordType = CoordType::Tool;
    std::vector<double> posCoord;
    std::vector<double> coord;
};
```

异常类

CodroidException / CodroidCommandException

```
class CodroidException : public std::runtime_error {
public:
    explicit CodroidException(const std::string& message);
};

class CodroidCommandException : public CodroidException {
public:
    int request_id() const noexcept;
    const std::string& command_ty() const noexcept;
    const std::string& controller_error() const noexcept;
    const std::string& raw_response_json() const noexcept;
};
```

固件版本常量

```
inline constexpr const char* MinControllerFirmware = "2.3.3.43";
```


SendCommand

```
void SendCommand(const std::array<double, 6>& position6, TrajectorySpace space);
```

SendTrajectory

```
void SendTrajectory(const std::vector<TrajectoryPoint>& trajectory, TrajectorySpace space,
int period_ms,
                const std::atomic<bool>* cancel = nullptr);
```

Close / IsOpen

```
void Close() noexcept;
bool IsOpen() const noexcept;
```

TrajectoryGenerator

离线轨迹生成。

```
#include "Codroid/trajectory_generator.hpp"
```

Generate

```
static std::vector<TrajectoryPoint> Generate(const std::array<double, 6>& start,
                const std::array<double, 6>& target,
                const TrajectoryRequest& request);
```

GenerateMultiSegment

```
static std::vector<TrajectoryPoint> GenerateMultiSegment(const
std::vector<std::array<double, 6>>& waypoints,
                const TrajectoryRequest& request);
```

TrajectoryRequest

```
struct TrajectoryRequest {
    TrajectorySpace space = TrajectorySpace::Joint;
    TrajectoryProfile profile = TrajectoryProfile::Trapezoidal;
    double duration_s = 1.0;
    double frequency_hz = 250.0;
    double max_velocity = 0.0;
    double max_acceleration = 0.0;
    double max_jerk = 0.0;
};
```

TrajectorySpace / TrajectoryProfile

```
enum class TrajectorySpace { Joint, Cartesian };
enum class TrajectoryProfile { Cubic, Trapezoidal };
```

完整示例

```
#include "Codroid/client.hpp"
#include "Codroid/cri_realtime_dispatcher.hpp"
#include "Codroid/trajectory_generator.hpp"

// 1. 连接
Codroid::CodroidClient robot;
robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150");

// 2. 启动 CRI 推送
robot.StartCriDataPush("192.168.8.150", 9030);
robot.WaitForCriData(5.0);

// 3. 启动实时控制
robot.StartCriControl(1, 4, 5);

// 4. 等待实时控制模式生效
while (!robot.GetRobotRealtimeState().realtime_control_mode) {
    std::this_thread::sleep_for(std::chrono::milliseconds(10));
}

// 5. 生成轨迹
auto state = robot.GetRobotRealtimeState();
std::array<double, 6> start;
for (int i = 0; i < 6; i++) start[i] = state.joint_position[i];

Codroid::TrajectoryRequest request;
request.space = Codroid::TrajectorySpace::Joint;
request.duration_s = 2.0;
request.frequency_hz = 250;

auto trajectory = Codroid::TrajectoryGenerator::Generate(
    start, {10, 0, 90, 0, 90, 0}, request);

// 6. 下发轨迹
Codroid::CriRealtimeDispatcher dispatcher("192.168.8.136", 9030, true);
dispatcher.SendTrajectory(trajectory, Codroid::TrajectorySpace::Joint, 4);

// 7. 清理
robot.StopCriControl();
robot.StopCriDataPush();
robot.Disconnect();
```

IO 与寄存器 API 参考

数字量 IO

GetDi / GetDo

```
int GetDi(int port, int id = 1);
int GetDo(int port, int id = 1);
```

SetDo

```
CommandResult SetDo(int port, int value, int id = 1);
```

模拟量 IO

GetAi / GetAo

```
double GetAi(int port, int id = 1);
double GetAo(int port, int id = 1);
```

SetAo

```
CommandResult SetAo(int port, double value, int id = 1);
```

寄存器

GetRegisterValue / GetRegisterValues

```
double GetRegisterValue(int address, int id = 1);
std::vector<ClientRegisterInfo> GetRegisterValues(const std::vector<int>& addresses, int id = 1);
```

SetRegisterValue

```
CommandResult SetRegisterValue(int address, double value, int id = 1);
```

完整示例

```
#include "Codroid/client.hpp"

Codroid::CodroidClient robot;
robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150");
```

```
// IO 操作
int di0 = robot.GetDi(0);
robot.SetDo(10, di0);

double ai0 = robot.GetAi(0);
robot.SetAo(0, 3.3);

// 寄存器操作
double value = robot.GetRegisterValue(49100);
robot.SetRegisterValue(49100, value + 1);

// 批量读取
std::vector<int> addresses = {49100, 49101, 49102};
auto values = robot.GetRegisterValues(addresses);

robot.Disconnect();
```

辅助工具 API 参考

主题订阅

SubscribePublishTopic

```
ClientPublishSubscription SubscribePublishTopic(  
    std::string topicTy,  
    std::function<void(const ClientPublishNotification&)> handler,  
    int tc_milliseconds = 100);
```

可用主题

主题	说明
publish/ProjectState	工程运行状态
publish/VarUpdate	变量更新
publish/RobotStatus	机器人状态
publish/RobotPosture	机器人位姿
publish/RobotCoordinate	坐标系
publish/Log	日志
publish/Error	错误

PublishTopics 常量

```
struct PublishTopics {  
    static constexpr const char* ProjectState = "publish/ProjectState";  
    static constexpr const char* VarUpdate = "publish/VarUpdate";  
    static constexpr const char* RobotStatus = "publish/RobotStatus";  
    static constexpr const char* RobotPosture = "publish/RobotPosture";  
    static constexpr const char* RobotCoordinate = "publish/RobotCoordinate";  
    static constexpr const char* Log = "publish/Log";  
    static constexpr const char* Error = "publish/Error";  
};
```


全局变量

GetGlobalVars / SaveGlobalVars / RemoveGlobalVars

```
nlohmann::json GetGlobalVars(int id = 1);
CommandResult SaveGlobalVars(const std::map<std::string, Variable>& vars, int id = 1);
CommandResult RemoveGlobalVars(const std::vector<std::string>& names, int id = 1);
```

运动学

ForwardKinematics / InverseKinematics

```
std::vector<double> ForwardKinematics(const FKParams& params, int id = 1);
std::vector<double> InverseKinematics(const IKParams& params, int id = 1);
```

CalculateRelativePose

```
std::vector<double> CalculateRelativePose(const RelativePoseParams& params, int id = 1);
```

控制台 UTF-8

ConsoleUtf8

```
#include "Codroid/console_utf8.hpp"

// Windows 控制台 UTF-8 支持
#ifdef _WIN32
Codroid::ConsoleUtf8::InitConsoleUtf8();
#endif
```

完整示例

```
#include "Codroid/client.hpp"

// 主题订阅
auto sub = robot.SubscribePublishTopic(
    Codroid::PublishTopics::RobotStatus,
    [](const Codroid::ClientPublishNotification& n) {
        std::cout << "状态: " << n.db_json << std::endl;
    }
);

// 全局变量
robot.SaveGlobalVars({
    {"counter", Codroid::Variable(42, "计数器")},
```

```
    {"name", Codroid::Variable(std::string("test"), "名称")}
});

// 运动学
Codroid::FKParams fk({0, 0, 90, 0, 90, 0});
auto tcp = robot.ForwardKinematics(fk);

Codroid::IKParams ik({400, 0, 300, 180, 0, 0});
ik.rj = {0, 0, 90, 0, 90, 0};
auto joints = robot.InverseKinematics(ik);
```